# Distributed Commitment Escrow

*A commitment nobody can see.*

*That cannot be taken back.*

*And that fires all at once, or not at all.*

## The features

### Part 1: Locking a commitment

Each company locks its commitment in an opaque envelope: everyone can see it exists, nobody can see what is inside, and the content cannot be swapped afterwards.

#### How it works

- The company picks a secret value (what it commits to) and a random number that hides it.
- A Pedersen commitment is computed from both: C = g^v · h^r, on an elliptic curve with fixed, agreed g and h.
- C is published. It reveals nothing about the value (hiding), and no other value can produce the same C (binding).
- The value and the random number stay with the company until the reveal.

#### What it defends against

**Threat:** A company changes its mind after seeing what the others did, and claims it committed to something else.

**Answer:** The locked commitment matches no other value, so any change is caught at the reveal.

#### Technologies and design decisions

| Component | Type |
|---|---|
| Pedersen Commitment | Existing technology |
| Elliptic-curve libraries (libsodium / py_ecc) | Existing technology |
| PostgreSQL | Existing technology |

#### Data structure: Commitment

commitment_id, company_id, commitment_value, submitted_at, purpose_tag

### Part 2: Coordinated reveal

The envelopes open only when the threshold is reached, all together, and only after every reveal has been checked.

#### How it works

- The coordinator counts the commitments received.

- Once the preset threshold (M of N) is reached, a reveal request goes to all participants at once.
- Each company reveals its value and random number. Anyone can check that the computation gives the original commitment.
- Only when every reveal is verified is the pause triggered.

### What mistake it prevents

**Threat:** A company reveals a convenient value after seeing the others, or refuses to reveal.

**Answer:** Simultaneous reveal and public verification. A mismatch or a refusal cancels the round.

### Technologies and design decisions

| Component | Type |
|---|---|
| M-of-N threshold | Our decision |
| Hash Time-Locked Contract (HTLC) | Existing technology |
| Pedersen Commitment | Existing technology |

### Data structure: RevealEvent

commitment_id, revealed_value, blinding_factor, verified

## Part 3: Public log against double commitment

Everyone sees exactly the same list of commitments. A company cannot show one thing to one side and another to the other.

### How it works

- Each commitment is added to a public log built as a Merkle tree: one fingerprint for the whole list.
- After every addition the tree head is signed and published.
- Participants compare the head each one received (gossip).
- If two sides received different heads, there is proof that someone split the log.

### What it defends against

**Threat:** A server or company shows each side a different version of the list (equivocation).

**Answer:** Comparison between participants exposes a split immediately, because the fingerprints will not match.

### Technologies and design decisions

| Component | Type |
|---|---|
| עץ Merkle + חתום ראש (STH) | Existing technology |
| Ed25519 signature on the tree head | Existing technology |
| Cross-participant consistency check (Gossip) | Built from scratch |
| CONIKS (מאמר מחקר) | Existing technology |
| S3 Object Lock (undeletable backup) | Existing technology |

### Data structure: LedgerEntry

leaf_hash, merkle_root_after, signed_tree_head, timestamp

## Part 4: Time window and automatic return

Whoever locked and vanished does not freeze everyone. When time runs out, everything is cancelled and returned.

### How it works

- Each round has a time window, for example 72 hours, in which the threshold must be reached.
- If the threshold is reached, the reveal begins.
- If time runs out first, all commitments are cancelled automatically, with nobody’s consent.
- Any company can submit again in a new round.

### What mistake it prevents

**Threat:** A company locks and disappears, and the system waits forever.

**Answer:** Window expiry and automatic return, inspired by the refund path of an HTLC.

### Technologies and design decisions

| Component | Type |
|---|---|
| Time window with states | Our decision |
| Hash Time-Locked Contract (HTLC) | Existing technology |
| PostgreSQL | Existing technology |

### Data structure: CommitmentWindow

window_id, opened_at, expires_at, required_threshold, status

## Part 5: Protection against misuse

The mechanism is technically limited to one purpose: pausing a safety risk. Not price fixing, not anything else.

### How it works

- Every commitment must carry a purpose tag from a closed, approved list, never free text.
- A neutral party (not a commercial one) signs every new tag that enters the list.
- Participant identities are public from the first moment. Only the content of the commitment stays locked.
- Sticking technically to the approved purpose is what will later allow reliance on a legal Safe Harbor.

### What it defends against

**Threat:** Companies use the mechanism to fix prices or split markets, instead of pausing a safety risk.

**Answer:** Only an approved purpose gets in and participants are visible, so a secret club cannot form.

### Technologies and design decisions

| Component | Type |
|---|---|
| Closed purpose list | Our decision |
| Neutral-party key (Regulatory Observer Key) | Built from scratch |
| Safe Harbor (legal pattern) | Our decision |

### Data structure: ApprovedPurposeTag

tag_id, tag_name, description, approved_by

## Part 6: Scaling to N parties

Not just two companies. Any number, with a flexible threshold, and no single party that everyone must trust.

### How it works

- An M-of-N threshold is set, for example four of six labs.
- Each company feeds its input into a joint computation (MPC) without revealing it.
- The computation answers one question: did we reach the threshold?
- The system keeps working correctly even if fewer than half of the participants drop out or lie.

### What mistake it prevents

**Threat:** A single trusted third party that becomes a point of failure.

**Answer:** Joint computation with no trusted party, resilient to disruptive participants.

### Technologies and design decisions

| Component | Type |
|---|---|
| MP-SPDZ | Existing technology |
| M-of-N threshold | Our decision |

### Data structure: MpcSession

session_id, participants, threshold_m, result

## Part 7: Proving participation without revealing

Anyone outside can verify that at least K companies committed, without knowing which, and without knowing to what.

### How it works

- A computational circuit is built that checks: "at least K valid values exist in the log".
- A short proof (zk-SNARK) confirms the claim without revealing the values.
- The proof is published with the public inputs: the threshold and the tree root.
- Anyone can verify it in seconds, with no interaction with the prover.

### What mistake it prevents

**Threat:** Full publication of the commitments hurts whoever started, and total silence gives the public no trust.

**Answer:** Partial transparency: the public knows the threshold was reached, and participants stay protected.

### Technologies and design decisions

| Component | Type |
|---|---|
| circom + snarkjs | Existing technology |
| עץ Merkle + ראש חתום (STH) | Existing technology |

### Data structure: ZkParticipationProof

proof_id, window_id, proof_blob, public_inputs, verified

## Six decisions that could have gone the other way

**A simple hash or Pedersen?** We chose Pedersen. The random number hides even a value with few possibilities, and the algebraic structure suits zero-knowledge proofs later. Price: elliptic-curve computation, and dependence on an audited cryptographic library.

**A trusted coordinator or one everyone audits?** We chose a coordinator everyone audits. The coordinator counts and triggers, but every result can be verified independently from the public log. Nobody has to be taken on trust. Price: every participant must run checks, not just receive an answer.

**Free text or a closed purpose list?** We chose a closed list. Only pre-approved tags get in, and a neutral party signs each one. This limits the mechanism to what it was built for. Price: less flexibility, and an approval process for every new purpose.

**No time limit or a 72-hour window?** We chose a fixed time window. When the window ends without the threshold, everything is cancelled and returned. No company stays bound to a promise nobody answered. Price: a failed round must start over.

**Reveal one by one or all together?** We chose a simultaneous reveal. Once the threshold is reached, everyone reveals at the same moment. Nobody sees the others' cards before showing their own. Price: one reveal moment must be coordinated for everyone.

**An open list or a zero-knowledge proof?** We chose a zero-knowledge proof. We prove to the public that the threshold was reached without showing who committed. Public trust does not come at the expense of those who dared to start. Price: a complex proof circuit that must be built and tested.

## What the first version taught us

| First version | Specification |
|---|---|
| Two mock nodes only | Any number of companies, M-of-N threshold |

| First version | Specification |
| --- | --- |
| A simple Pedersen commitment | A signed public log that prevents sending different commitments to different parties |
| We did not handle a company that vanishes | A time window and an automatic return path |
| We assumed someone operates the system | Anyone can verify, with no single trusted authority |
| We did not handle misuse | An approved purpose from a closed list, and public participant identities |